

Functions and packages

Computational Methods I

Course WBPH064-05

Lecture 3



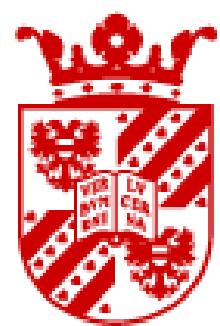
**university of
 groningen**

**faculty of science
 and engineering**

**zernike institute for
 advanced materials**

jagoda.slawinska@rug.nl

1. Functions
2. Built-in functions
3. Methods
4. Defining functions and documenting them
5. Recursions
6. Packages
7. Numpy



Function: a piece of code that only runs once it is called.

Input: argument(s) that you provide to run the function

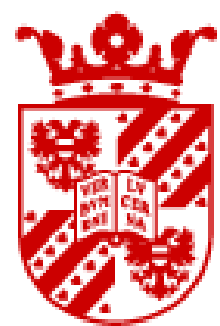
Output: value(s) that will be returned by the function

```
1 return_value = function(argument)
```

```
1 return_value = function(argument1, argument2)
```

```
1 my_string = "Computational Methods 1"  
2 type(my_string)
```

```
str
```



Functions vs. Methods

Built-in functions: Functions that are always available.

Examples:

```
my_list = [1,2,3,4,5]
```

```
type(my_list)
```

list

```
print(my_list)
```

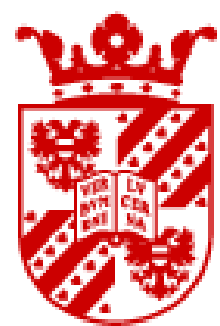
[1, 2, 3, 4, 5]

```
len(my_list)
```

5

```
input('Provide input:')
```

Provide input: input
'input'



university of
 groningen

faculty of science
and engineering

zernike institute for
advanced materials

jagoda.slawinska@rug.nl

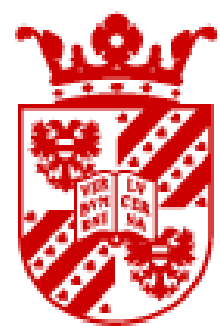
Methods: a method is a function that is associated with an object

- Directly act (and change) the object.
- Typically faster, but less flexible

Example: `list.append()`:

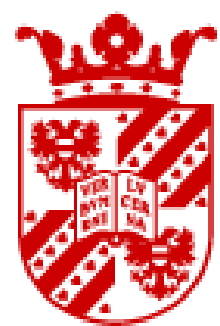
```
my_list = [1,2,3,4,5]  
my_list.append(6)  
print(my_list)
```

```
[1, 2, 3, 4, 5, 6]
```



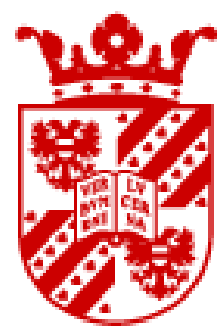
Function definition

```
def my_function(arg1, arg2):  
    """This is the docstring, that explains what the function does."""  
  
    # Do something using the arguments to produce the output(s)  
  
    return output1, output2
```



Functions can return different outputs based on conditions

```
def my_function(arg1, arg2):  
    """This is the docstring, that explains what the function does."""  
  
    # Do something using the arguments to produce the output(s)  
  
    if arg1 > arg2:  
        return output1  
    else:  
        return output2
```



Docstrings

- Documentation string

- Style guide:

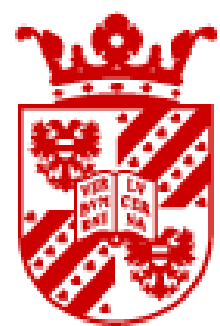
<https://peps.python.org/pep-0257/>

```
def add(a, b):  
    """Add the numbers a and b."""  
    return a + b
```

```
help(add)
```

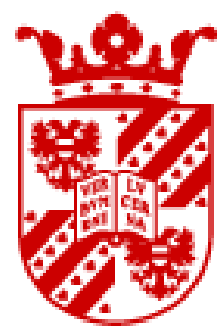
Help on function add in module __main__:

```
add(a, b)  
    Add the numbers a and b.
```



Functions in functions

```
def func1(arg1, arg2):  
    """Function that uses another function"""  
  
    #Function that takes arg2 as input  
    out2 = func2(arg2)  
  
    #Computation of arg1 and out2 combined  
    output = arg1*out2  
  
    return output
```



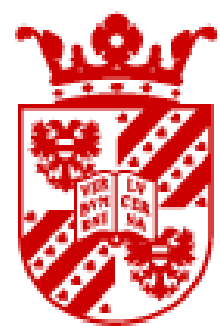
Recursion: function that calls itself

- Base case:

The recursion is terminated (the function stops calling itself)

- Each recursion gets you closer to the base case

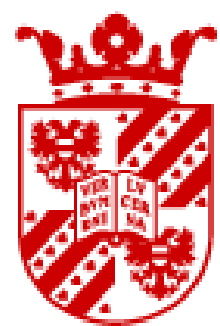
The base must be reachable



Example, countdown by recursion

```
def countdown(n):  
    """Recursive function that counts down from a positive n to zero."""  
  
    #Print current value of n  
    print(n)  
  
    #Stop if n equals zero, continue otherwise  
    if n == 0:  
        return None  
    else:  
        return countdown(n-1)  
  
countdown(2)
```

2
1
0



Arguments

- Positional arguments

Function requires these to run

- Keyword arguments

Defined after the positional arguments

Have a default value

Positional argument

Keyword argument

```
def add(a, b=1):  
    """  
    Add two integers and return the result.  
  
    Parameters:  
    a (int): The first integer to add.  
    b (int): The second integer to add (default is 1).  
  
    Returns:  
    int: The sum of the two input integers.  
    """  
    return a + b
```



Scope of the variable

- Local variable:
Inside a function (can't be accessed outside)
Is destroyed after function executed
- Global variable:
Outside functions
Can be accessed always (inside & outside functions)

Global variable

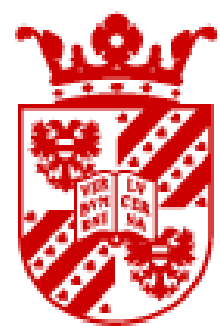
Local variable

```
a = 3

def function():
    a = 10
    print(a)

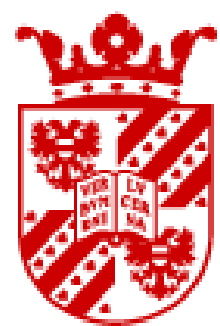
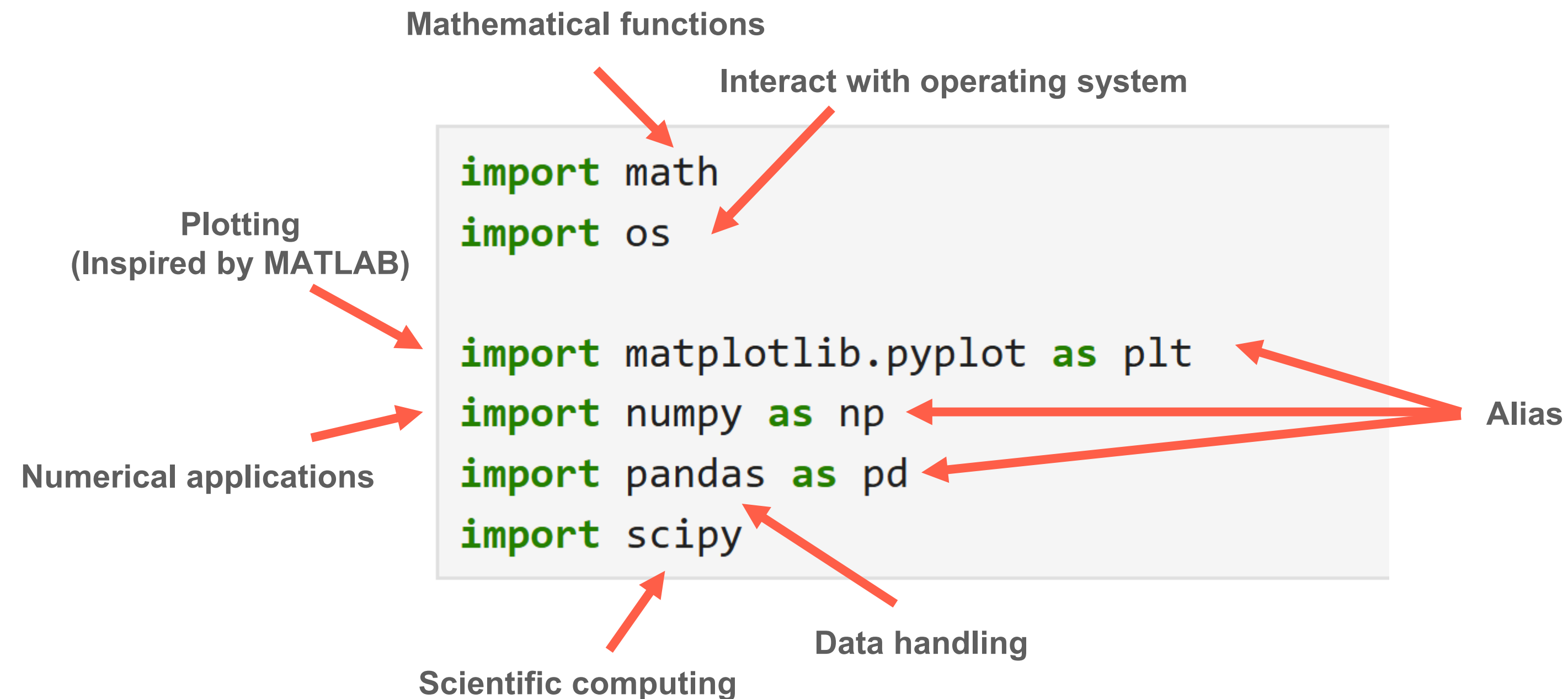
function()
print(a)
```

10
3



Import code you don't have to write yourself

- Common functions
- Optimized



NumPy

- Numerical Python
- Numerical applications
- Documentation

<https://numpy.org/doc/stable/index.html>

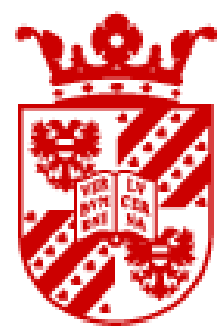
```
import numpy as np
```

```
matrix = np.array([[1, 2], [3, 4]])  
print(matrix)
```

```
[[1 2]  
 [3 4]]
```

```
np.dot(matrix, matrix)
```

```
array([[ 7, 10],  
       [15, 22]])
```



university of
 groningen

faculty of science
 and engineering

zernike institute for
 advanced materials

jagoda.slawska@rug.nl